Selector-Based TCP Server

Visual guide: module boundaries, thread ownership, and the request-response round trip

1. Core idea

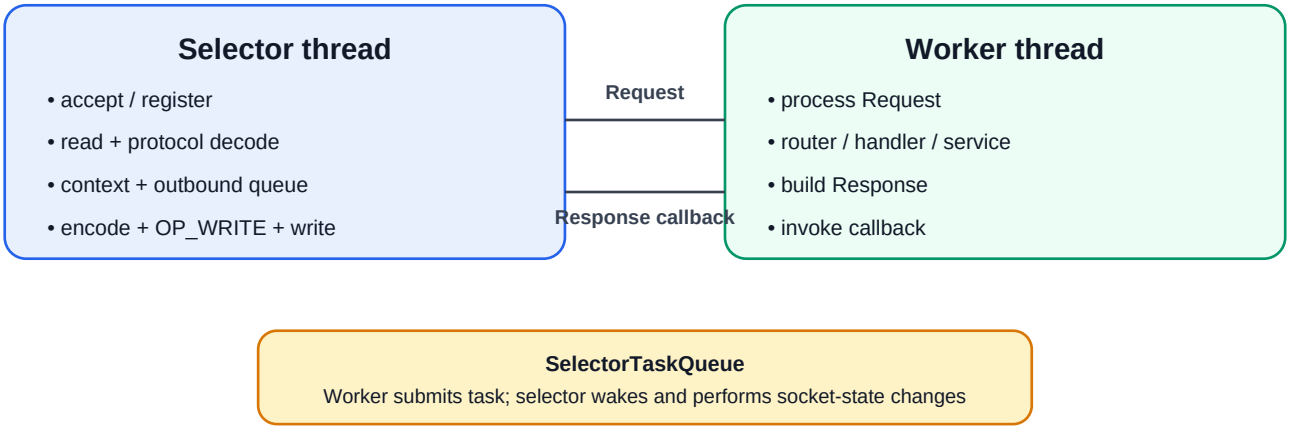
The selector thread owns sockets and mutable connection state. Worker threads own application processing. Only decoded Requests go to workers; only Responses return.

2. Module boundaries

Module	Owns	Must not know
server-contracts	Request, Response, processing/dispatch/callback contracts	TCP, bytes, workers, business rules
protocol-spi	ProtocolSession contracts	Selector, sockets, handlers
protocol-line	Partial input, newline framing, encode/decode	SocketChannel, worker pool
application-core	Router, handlers, services, commands/responses	Selector, ByteBuffer, ExecutorService
server-execution	WorkerPoolRequestDispatcher; scheduling	Protocol syntax, sockets, concrete router
transport-tcp-selector	Selector, channels, keys, context, writes	Business rules
server-runtime	Creates and wires implementations	Reusable behavior

Dependency rule: server-execution depends only on server-contracts. server-runtime injects RequestRouter as RequestProcessor.

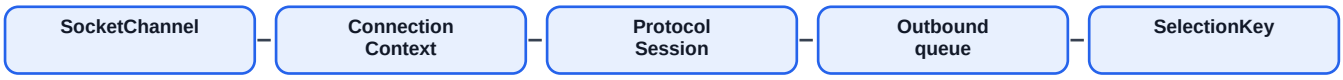
3. Thread ownership



Rule: workers never mutate ConnectionContext, SelectionKey, SocketChannel, or ProtocolSession directly.

4. One connection = one ConnectionContext

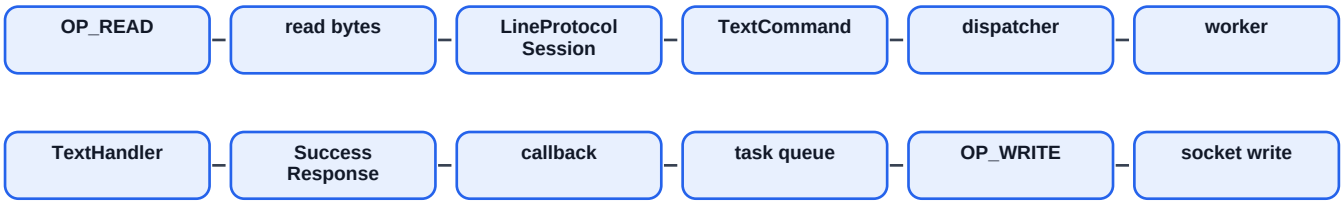
Every accepted client gets a separate context containing its ProtocolSession, outbound ByteBuffer queue, and SelectionKey.



TCP may split input. Client A can send **TEXT|hel**, then later **lo\n**. Its own protocol session remembers the partial text. Client B has a separate session, so bytes never mix.

5. Worked request-response example

Client sends **TEXT|hello\n**.



1. Selector reads bytes. 2. Protocol session creates TextCommand. 3. Dispatcher submits to worker. 4. Router invokes TextHandler. 5. Worker invokes callback. 6. Callback submits selector task and wakes selector. 7. Selector encodes response, calls **enqueueOutbound(buffer)**, enables OP_WRITE. 8. handleWrite drains the queue.

6. Outbound queue and partial writes

A socket may write only part of a response. The ByteBuffer stays queued until no bytes remain. Then it is removed. When the queue becomes empty, remove OP_WRITE to avoid a busy loop.

```
OK|Echo: hello\n → first write may send only OK|Echo: → buffer remains queued → next
OP_WRITE sends the rest
```

7. Safe worker-to-selector handoff

```
worker Response → SelectorTaskQueue.submit(task) → selector.wakeup() → selector runs task
→ encode + enqueueOutbound + enable OP_WRITE
```

This preserves one owner for mutable NIO state and avoids races and cancelled-key failures.

8. Protocol examples

```
PING\n → OK|PONG\n
TEXT|hello\n → OK|Echo: hello\n
STUDENT_CREATE|Avinash|25\n → OK|Student created: Avinash, age=25\n
```

Next production concerns: ordering per connection, worker rejection, protocol errors, queue limits/backpressure, timeouts, graceful shutdown, logging, metrics, and TLS.